

内存安全与线程安全

安全背景

在安全关键系统中，内存损坏可能导致不可预测的行为，进而引发系统故障。每一次内存分配、每一个共享变量、每一次缓冲区访问都必须经过验证。

堆内存规则

规则：大型局部变量应评估栈使用情况

嵌入式平台的栈空间有限。线程中过大的局部变量会导致栈溢出风险。当栈空间紧张时，应使用堆分配代替大型局部变量。

```
/* 反面示例 - 线程中在栈上分配256字节 */
void worker_thread(void *arg)
{
    uint8_t buffer[256];
    /* ... */
}

/* 正面示例 - 使用堆分配 */
void worker_thread(void *arg)
{
    uint8_t *buffer = (uint8_t *)platform_malloc(256);
    if (buffer == NULL) {
        error_handler(ERR_OUT_OF_MEMORY);
        return;
    }
    /* ... 使用 buffer ... */
    platform_free(buffer);
}
```

非标准堆API

嵌入式平台通常使用自定义内存分配器，而非标准的 `malloc / free`。在编写任何堆分配代码之前：

1. 搜索项目中现有的分配模式
2. 查找 `platform_malloc`、`pvPortMalloc`、`os_malloc`、`mem_alloc` 或其他项目特定的函数
3. 使用与项目中其他文件完全相同的API
4. 绝不混用不同的分配器族

分配/释放规范

1. **检查每次分配** - 始终检查NULL返回值
2. **每次分配都要配对释放** - 清晰地文档化所有权
3. **按相反顺序释放** - 如果A分配了B、B分配了C，则先释放C再释放B再释放A
4. **释放后置空** - 释放后将指针设为NULL
5. **错误路径清理** - 发生错误时，释放此前已分配的所有资源

错误路径模式

```
int module_init(Module_t *self)
{
    self->buf_a = platform_malloc(BUF_A_SIZE);
    if (self->buf_a == NULL) {
        return ERR_NO_MEMORY;
    }

    self->buf_b = platform_malloc(BUF_B_SIZE);
    if (self->buf_b == NULL) {
        goto fail_buf_b;
    }

    self->queue = queue_create(Queue_DEPTH);
    if (self->queue == NULL) {
        goto fail_queue;
    }

    return ERR_OK;

fail_queue:
    platform_free(self->buf_b);
    self->buf_b = NULL;
fail_buf_b:
```

```
platform_free(self->buf_a);
self->buf_a = NULL;
return ERR_NO_MEMORY;
}
```

线程安全

共享资源保护

任何被多个线程访问或在ISR与线程之间共享的数据都必须受到保护：

场景	保护机制
线程 <-> 线程	互斥锁 / 信号量
ISR <-> 线程	临界区 / 关中断
读多写少的数据	读写锁（如可用）
简单标志位	原子操作
数据交换	消息队列（推荐）

消息队列模式（推荐）

消息队列是不同上下文之间交换数据最安全的方式。它完全避免了共享状态：

```
typedef struct {
    uint8_t cmd;
    uint16_t data_len;
    uint8_t data[MAX_MSG_DATA_SIZE];
} DriverMsg_t;

/* 生产者（应用线程） */
int Driver_SendAsync(Driver_t *self,
                    const uint8_t *data,
                    uint16_t len)
{
    DriverMsg_t msg;
    msg.cmd = CMD_SEND;
    msg.data_len = len;
    memcpy(msg.data, data, len);
    return queue_send(self->msg_queue, &msg, TIMEOUT_MS);
}
```

```

}

/* 消费者（驱动工作线程） */
static void driver_worker(void *arg)
{
    Driver_t *self = (Driver_t *)arg;
    DriverMsg_t msg;

    while (self->running) {
        if (queue_recv(self->msg_queue, &msg,
                       POLL_INTERVAL_MS) == OK) {
            process_message(self, &msg);
        }
    }
}

```

临界区规则

1. 临界区要尽可能短
2. 绝不在临界区内调用阻塞函数
3. 绝不在临界区内分配内存
4. 文档标明为什么需要临界区

可重入性

如果一个函数可以安全地被多个线程同时调用，或者在第一次调用 完成之前被中断并再次调用，则该函数是可重入的。

如何使函数可重入

1. 不使用静态/全局可变量
2. 不调用不可重入的函数
3. 所有状态通过参数传递
4. 使用局部变量（在栈上，注意大小限制）或调用者提供的缓冲区

文档标注可重入性

```

/**
 * @brief 计算数据缓冲区的CRC32。
 * @note 本函数可重入且线程安全。
 *       不使用任何静态或全局状态。

```

```
*/
uint32_t crc32_calc(const uint8_t *data, uint32_t len);

/**
 * @brief 通过UART端口发送数据。
 * @note 不可重入。调用者必须通过驱动的公共API
 *       序列化访问，驱动内部处理同步。
 */
static int uart_hw_send(const uint8_t *data, uint16_t len);
```

栈溢出防护

风险因素

- 线程中的大型局部数组/结构体（需评估栈空间是否充裕）
- 深度递归（尽量完全避免递归）
- 过深的函数调用链
- VLA（变长数组）——禁止使用

缓解措施

1. 线程中使用堆分配大型缓冲区
2. 避免递归，使用迭代算法
3. 测试时监控栈高水位标记
4. 根据实际测量用量加余量配置栈大小
5. 如RTOS提供栈溢出检测钩子，则启用

缓冲区溢出防护

1. 始终将缓冲区大小与缓冲区指针一起传递
2. 复制操作前验证输入长度
3. 使用带有明确大小的 `memcpy`，绝不使用 `strcpy`
4. 优先使用 `snprintf` 而非 `sprintf`
5. 缓冲区大小定义为宏，绝不使用魔法数字

```
#define RX_BUF_SIZE (256U)
```

```
/* 正面示例 */
```

```
int process_data(const uint8_t *src, uint16_t src_len,
                uint8_t *dst, uint16_t dst_size)
{
    if (src_len > dst_size) {
        return ERR_BUFFER_TOO_SMALL;
    }
    memcpy(dst, src, src_len);
    return ERR_OK;
}
```

防御性编程

断言 vs 运行时检查

根据不同情况使用正确的工具：

情况	机制
程序员错误（不应该发生）	<code>ASSERT()</code>
外部/不可信输入	运行时检查 + 错误码
硬件故障检测	运行时检查 + 恢复机制

```
/* 断言用于内部契约 */
void Module_Process(Module_t *self)
{
    ASSERT(self != NULL);
    ASSERT(self->state != STATE_UNINITIALIZED);
    /* ... */
}

/* 运行时检查用于外部输入 */
int Module_SetValue(Module_t *self, int32_t value)
{
    if (value < VALUE_MIN || value > VALUE_MAX) {
        return ERR_INVALID_PARAM;
    }
    self->value = value;
    return ERR_OK;
}
```

防御性编码规则

1. **验证所有指针参数** 后再解引用
2. **验证数组索引** 后再访问
3. **验证枚举值** ——不要假设它们在范围内
4. **检查每个函数调用的返回值**
5. **初始化所有变量** ——不要依赖默认值
6. **在switch中使用 `default`** ——即使枚举已覆盖所有值（未来可能添加新值）

详细的防御性编程模式（包括const正确性和错误处理策略），参见 `03_Clean_Code.md`。

获取更多

- **B站搜索「兆鸣嵌入式」** 观看完整教程（详细讲解+代码实战）
- 公众号搜索「兆鸣嵌入式」回复「交流」加技术交流群
- 抖音搜索「兆鸣嵌入式」看短视频精华版